

Mixed-Precision Attention on Consumer Blackwell

A Pareto Replication Study from FP32 to NVFP4 on the RTX 5080

Refik Can Öztaş

N25142279

rne

Ali Talip Coşkun

N25123060

Refik Can Öztaş

N25142279

Hacettepe University
Computer Engineering

Ali Talip Coşkun

N25123060

Hacettepe University
Computer Engineering

TL;DR

Five CUDA attention kernels, one consumer GPU (RTX 5080), and one research question:
do FP8 / FP4 wins from the literature actually transfer to mid-tier consumer Blackwell?

1. Why this matters

LLM cost,
GPU cycles, attention

2. What's been done

FlashAttention, FP8,
NVFP4 – a quick tour

3. Our gap

Why RTX 5080 is
unstudied; our question

4. The hardware

What the 5080 gives,
what it doesn't

5. The five-step ladder

V0 → V4, one
design change per rung

6. Findings + take-aways

Two wins, one
informative loss

Stop us anywhere with questions – this is a course presentation, not a TED talk.

Part 1 of 6

Why this matters

*The cost of running LLMs –
and where attention sits in that bill.*

Model size has grown $\sim 1000\times$ in 5 years

- ▶ GPT-2 (2019): **1.5 B** parameters
- ▶ GPT-3 (2020): **175 B**
- ▶ PaLM (2022): **540 B**
- ▶ GPT-4 (2023, est.): **$\sim 1.7 \text{ T}$**
- ▶ Frontier models 2025: trillions, mixture-of-experts

Watch out!

Both training *and* serving are GPU-bound:

- ▶ GPT-3 training: $\sim \$5 \text{ M}$ of compute
- ▶ GPT-4 training: $\sim \$100 \text{ M}$
- ▶ ChatGPT serving: $\sim \$700 \text{ K / day}$ (2023 est.)

Aha moment

Every LLM dollar is a GPU cycle. Cycles get spent inside a small set of operations: matmuls, attention, normalisations, activations, KV-cache I/O.

Of these, **attention scales worst with context length** – and context length is the axis everyone is racing on.

Shave 30 % off the attention kernel and you shave a measurable fraction off the cost of running the whole model.

Forward-pass cost breakdown (per transformer block)

- ▶ **Linear projections** (Q, K, V, output, MLP):
cost is $O(N \cdot D^2)$ – linear in N .
- ▶ **Attention** ($QK^\top$, softmax, PV):
cost is $O(N^2 \cdot D)$ – **quadratic in N** .
- ▶ **LayerNorm, activations**: small, often fused.
- ▶ **KV-cache I/O** (decode):
bandwidth-bound, scales with N .

Spoiler

For short prompts ($N \lesssim 2048$) the linear projections dominate. As N grows, attention takes over – because it grows as N^2 while everything else grows as N .

At $N = 32,768$ (GPT-4 default) attention is the *single largest* kernel in the forward pass on a typical decoder.

Why this matters for us. Models are racing toward 128K, 1M, 10M context windows. Attention's share of cost grows monotonically with that race. *Every vendor invests heavily here.*

The operation that made transformers work

For each query token q_i , score it against every key k_j , run a softmax over those scores, then use the resulting weights to mix the values v_j . The output for position i is a weighted sum over all values.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{D}}\right) V.$$

- ▶ Q, K, V : $N \times D$ matrices (N = sequence length, D = head dim).
- ▶ $S = QK^\top$: $N \times N$ – the score matrix; here the quadratic cost lives.
- ▶ Softmax is row-wise: each query distributes attention probability over all keys.
- ▶ PV : gives the output O , again $N \times D$.

Aha moment

“Each token reads every other token, decides how much it cares about each one, and pulls a weighted summary of their values back to itself.”

That all-to-all structure is exactly what makes attention powerful *and* what makes it expensive.

Two distinct problems, both rooted in the $N \times N$ score matrix.

Watch out!

Problem 1 – memory bandwidth.

S and P are each $N \times N$ per (batch, head). At $N = 8192$, batch 2, heads 8, FP32 that's ≈ 4.3 GB *per intermediate*. Reading and writing them through HBM at ~ 960 GB/s is the bottleneck – not the arithmetic.

Watch out!

Problem 2 – arithmetic throughput.

The two matmuls dominate FLOPs. They run fast *only if* data is in the right precision, the right layout, on the right units (Tensor Cores). FP32 ALU is $\sim 50\times$ slower than FP16 Tensor Cores; FP16 is $2\times$ slower than FP8; FP8 is $2\times$ slower than FP4.

Aha moment

Two levers, then. Avoid materialising S and P (*memory lever*), and use the narrowest precision Tensor Cores you can while keeping the output usable (*compute lever*). Our work measures both, in isolation, on the RTX 5080.

Part 2 of 6

What the field has done

*A short tour of FlashAttention,
FP8, and FP4 attention work.*

Thread A – algorithmic fusion

Restructure the computation so S and P *never* live in HBM. Same precision, less memory traffic.

- ▶ 2018 – Online softmax (Milakov & Gimelshein).
- ▶ **2022 – FlashAttention** (Dao et al., A100).
- ▶ 2023 – FlashAttention-2 (better partitioning).
- ▶ 2024 – FlashAttention-3 (Hopper, TMA, FP8).

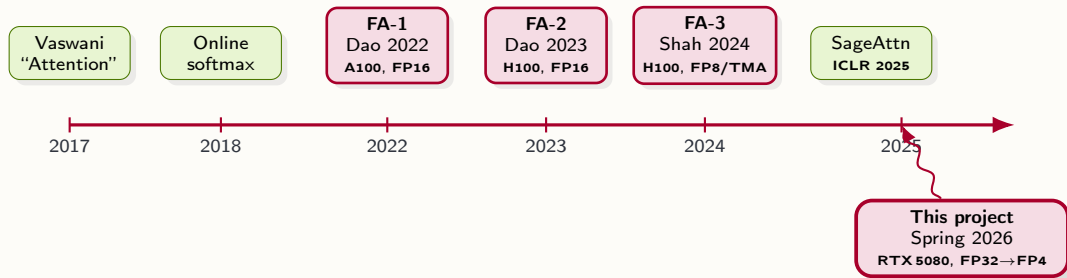
Thread B – lower precision

Carry the same computation in narrower data types so each Tensor-Core instruction does more work per cycle.

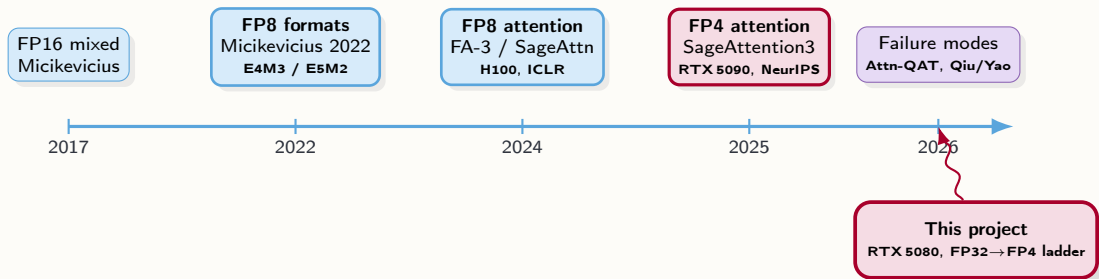
- ▶ 2017 – FP16 mixed precision (Micikevicius).
- ▶ **2022 – FP8 formats** (Micikevicius, NVIDIA).
- ▶ 2024 – FP8 attention deployed (FA-3, SageAttention).
- ▶ **2025 – NVFP4 attention** (SageAttention3, RTX 5090).

TL;DR

These threads are *multiplicative*, not exclusive. A modern optimised attention kernel applies fusion and narrow precision. Our ladder isolates the contribution of each thread on a



- ▶ Each step builds on the previous: tile loop, then better partitioning, then warp specialisation, then narrower precision.
- ▶ **Each was validated on the latest datacenter or flagship GPU of its time.** Mid-tier consumer cards came later, often by years.



- ▶ NVIDIA hardware leads precision innovations; software catches up 1–2 years later (FP8 hardware: Hopper 2022; FP8 attention paper: 2024).
- ▶ **NVFP4 (E2M1) is the newest member:** hardware on Blackwell (2024), first attention application on RTX 5090 (SageAttention3, 2025).

Hardware	FA-1	FA-2	FA-3 / FP8	NVFP4	Tier
A100 (Ampere)	✓	✓	—	—	datacenter
H100 (Hopper)	—	✓	✓	—	datacenter
B100 / B200 (Blackwell DC)	—	✓	✓	✓	datacenter
RTX 5090	—	—	✓	✓ (SageAttn3)	consumer flagship
RTX 5080	?	?	?	?	consumer mid-tier

What broke

The RTX 5080 shares its compute capability (sm_120a) with the RTX 5090, but **half** the SM count (84 vs 170), **half** the VRAM (16 GB vs 32 GB), and a narrower memory bus. *“It works on the 5090” does not imply “it works on the 5080.”* Yet the 5080 is the realistic target for individual researchers, small labs, and edge-deployed inference.

Research question

*How does the latency–accuracy–memory Pareto frontier of mixed-precision attention shape up on the RTX 5080 (sm_120a) as we add tiling, fusion, FP8, and NVFP4 **one variable at a time**?*

What we tried

A controlled ladder:

1. Build five hand-rolled CUDA kernels, V0 to V4.
2. Each rung changes *exactly one* design choice versus the previous rung.
3. Sweep all variants over the same grid; report latency, peak HBM, and rel-L2 accuracy.

4. Compare against PyTorch SDPA

Aha moment

Why a ladder, not just a final kernel:

Existing FP4 attention papers (e.g. SageAttention3) report only the final number. We can't tell from those whether the win comes from fusion, FP8, or FP4.

Our ladder makes each contribution measurable separately on identical hardware – and that is what surfaces our negative FP4 result.

Part 3 of 6

Meet the hardware

*What our consumer Blackwell card gives us
and what it stubbornly doesn't.*

NVIDIA GeForce RTX 5080 (our card)

- ▶ Architecture: **Blackwell**, `sm_120a`
- ▶ **84** streaming multiprocessors (SMs)
- ▶ **16 GB** GDDR7, ≈ 960 GB/s HBM
- ▶ 5th-generation Tensor Cores
- ▶ Native FP8 (E4M3 / E5M2) **and** FP4 (NVFP4)
- ▶ 256-bit memory bus
- ▶ Released early 2025

Aha moment

What it shares with datacenter Blackwell: same MMA instruction families, same FP8 / FP4 formats, the same NVFP4 hardware support *in principle*.

What broke

What it lacks (and why this hurts):

- ▶ No **TMA** (Tensor Memory Accelerator) – so FA-3's warp-specialised pipeline does not transfer.
- ▶ CUTLASS `77_blackwell_fmha` reference is gated to `sm_100a` / `sm_103a`.
- ▶ Hardware NVFP4 path requires

Same MMA instruction family, three precisions. The kernel selects one explicitly via inline PTX or via the high-level `nvcuda::wmma` API.

Precision	Mnemonic	Shape	Used by	Theory
FP16 (E5M10)	HMMA.16816.F32	$16 \times 8 \times 16$	V1, V2 (<code>wmma</code>)	1×
FP8 (E4M3)	QMMA.16832.F32.E4M3.E4M3	$16 \times 8 \times 32$	V3 (inline PTX)	2×
FP4 (E2M1)	QMMA.16832.F32.E2M1.E2M1	$16 \times 8 \times 32$	V4 (inline PTX)	4×

- ▶ **HMMA** = “**Half MMA**” – FP16 inputs, FP32 accumulator.
- ▶ **QMMA** = “**Quarter MMA**” on consumer Blackwell SASS; covers FP8 *and* FP4 (precision encoded in the suffix).
- ▶ Each instruction uses a per-thread *fragment* layout fixed by the PTX ISA. You load operands into registers, call the instruction, read the accumulator.
- ▶ FP4 specifically requires `sm_120a`, not plain `sm_120`.

FP8 (E4M3)

- ▶ 1 sign + 4 exponent + 3 mantissa bits
- ▶ Max representable magnitude: **448**
- ▶ ~ 16 representable positive values
- ▶ Reasonable for unscaled inputs around 1

Watch out!

FP4 (E2M1) – yes really, only this many values:

- ▶ 1 sign + 2 exponent + 1 mantissa bit
- ▶ Representable positives: **{0, 0.5, 1, 1.5, 2, 3, 4, 6}**
- ▶ Max magnitude: **6**
- ▶ **Useless without per-block rescaling.**

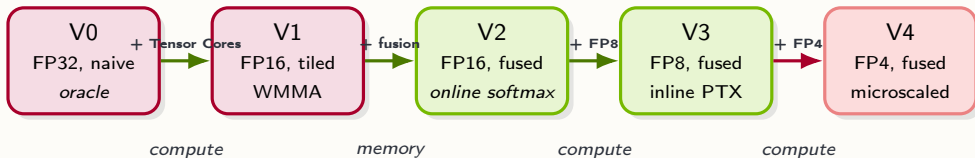
Aha moment

Microscaling, in plain language. Real activation tensors span many orders of magnitude. They will not fit into FP8 (let alone FP4) directly. *The trick:* divide each block (a tile, a row, or a small group of elements) by its own absmax, store the small-range result, multiply by the saved scale when reading back. The Tensor-Core instruction operates on

Part 4 of 6

The five- step ladder

*Five hand-rolled CUDA kernels.
One design change per rung.*



TL;DR

Each arrow changes *exactly one* design choice. The performance / accuracy delta at each step quantifies the contribution of that one choice on this hardware, with no confounding.

Spoiler

V0 → V3 are wins. V3 → V4 is an informative **loss**.

Motivation. We need a deliberately-slow but *transparently correct* baseline. Every other variant is checked against this kernel's output – without an oracle there is no way to claim a numerical regression.

What we tried

- ▶ Three separate kernel launches: QK^T , then row-wise softmax, then PV .
- ▶ All FP32; no Tensor Cores.
- ▶ Materialises full S and P in HBM.

What broke

Softmax row-reduce needs `__syncthreads()` *between* max-reduce and sum-reduce. Without it: a fast warp overwrites the shared row-max while a slow warp is still reading it. **6% non-deterministic mismatch on large N – hours**

The numbers

$N = 2048, D = 128$:

- ▶ Latency: **4,500** ms
- ▶ Memory: **640** MB
- ▶ Tensor-Core insns: **0**
- ▶ Rel L2 vs FP32: **0** (it is the reference)
- ▶ OOM past $N \approx 2048$

Aha moment

Take-away: naive attention is unworkable past tiny N . Optimisation isn't optional.

Motivation. Isolate the contribution of *Tensor Cores* + *tiling* before changing anything else. Keep V0's three-launch structure deliberately, so the V1→V2 step later cleanly measures fusion.

What we tried

- ▶ Inputs in FP16; FP32 accumulators.
- ▶ Both matmuls go through `nvcuda::wmma` – compiles to `HMMA.16816.F32`.
- ▶ **Trick #1:** get K^T for free by loading K as `matrix_b col_major`. Same memory, transposed in operand-space.
- ▶ **Trick #2:** all SMEM in a single dynamic allocation. Static + dynamic together hit the 48 KB per-block default cap at $D = 128$ – raise it via `cudaFuncSetAttribute`.
- ▶ Still materialises S and P in HBM (V2's

The numbers

$N = 8192$, $D = 128$:

- ▶ Latency: **77.1** ms
- ▶ Memory: **2176** MB
- ▶ HMMA insns: **120 + 32**
- ▶ Rel L2: 6×10^{-4}

$\sim 60\times$ faster than V0 already, just from Tensor Cores. But still allocates 2 GB.

Motivation – pull the memory lever

Intermediates S and P are what makes V1 allocate gigabytes. **FlashAttention's idea**: process keys / values in tiles, keep a running per-row max and denominator, never materialise the full S or P . *Same precision, smaller HBM footprint.*

$$\alpha^{(t)} = \exp(m^{(t-1)} - m^{(t)}), \quad O^{(t)} = \alpha^{(t)} O^{(t-1)} + \tilde{P}^{(t)} V^{(t)}.$$

What we tried

- ▶ One fused kernel; one block per output row-tile.
- ▶ Per-block SMEM at $D = 128$ totals **96 KB**, twice the default.
- ▶ Two boundary cases not in the FA paper but required: first iteration ($m = -\infty$), fully-masked rows. Both produce NaN

The numbers

The $17\times$ memory finding ($N = 8192$, $D = 128$):

V1 = 2176 MB $\rightarrow$ V2 = 128 MB

Same FP16 contract, no precision change. The full $N\times N$ S, P never appear in HBM.

Motivation – pull the compute lever, twice

Two contributions, both unlocked by writing inline PTX instead of using the `wmma` API: **FP8 mma throughput** (theoretical $2\times$ FP16) and a **register-resident output accumulator** (V2 round-tripped O through SMEM because WMMA fragment layout is implementation-defined; PTX layout is fully specified).

What we tried

- ▶ FP8 (E4M3) on Q, K, V, P. Inputs come in as FP16, quantised in-kernel.
- ▶ Per-tile FP8 scaling: $\text{scale} = \max(|x|)/448$ (Micikevicius 2022 recipe).
- ▶
`mma.sync.aligned.m16n8k32.row.col.f32.e4m3.`
via inline PTX.
- ▶ **Why not CUTLASS?** Its FMHA reference example is gated to `sm_100a` / `sm_103a`. No

The numbers

$N = 8192$, $D = 128$:

- ▶ Latency: **25.8 ms** ($2.4\times$ V2)
- ▶ Memory: **128 MB** (same as V2)
- ▶ QMMA insns: **96**
- ▶ Rel L2: 5×10^{-2}

Aha moment

Subt

Motivation – one more compute lever

FP4 promises another 2x over FP8. SageAttention3 has demonstrated this on RTX 5090 via the *hardware-microscaled* `mxvf4nvf4.m16n8k64` mma. **Does this transfer to RTX 5080?**

What we tried

- ▶ Same V3 skeleton, but FP4 (E2M1) instead of FP8.
- ▶ `kind::f8f6f4.m16n8k32` – the *software-microscaled* path. FP4 packed in the middle 4 bits of each byte.
- ▶ Per-row, per-K-block (block size 32) FP32 scales, applied at the accumulator after every mma.

What broke

Could not use mxf4nvf4.m16n8k64: its scattered-fragment layout requires 3d vector-style quizzled

The numbers

$N = 8192, D = 128$:

- ▶ Latency: **33.4 ms**
($0.77 \times V3$ – *slower!*)
- ▶ Memory: **128 MB**
- ▶ QMMA insns: **96**
- ▶ Rel L2: 0.21

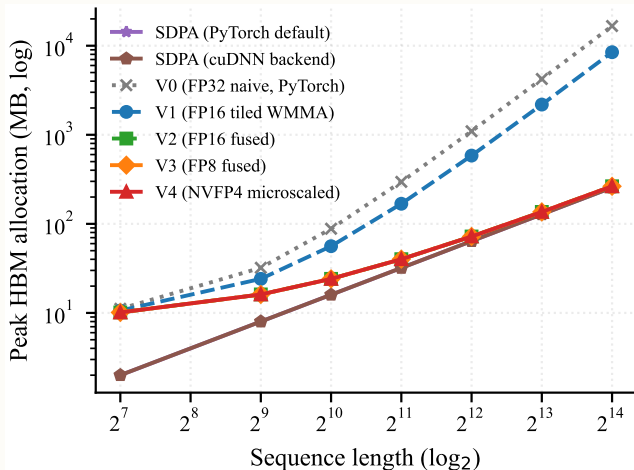
Spoiler

Software-microscaling overhead
> FP4 throughput advantage.
This is a finding, not a fail-

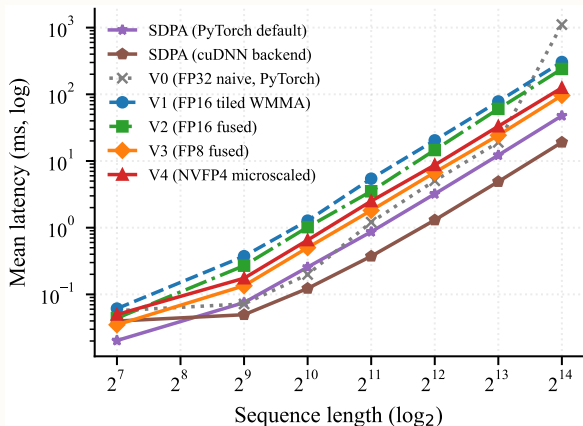
Part 5 of 6

Findings

*Two clean wins,
one informative loss.*



- ▶ V1 grows quadratically (full $N \times N$ in HBM); V2 grows linearly ($O(N \cdot D)$).
- ▶ At $N = 8192$, $D = 128$: V1 = **2176 MB**, V2 = **128 MB** $\Rightarrow$ **17 $\times$ reduction**.
- ▶ V2, V3, V4 and SDPA all overlap on this plot.



The numbers

$N = 8192$, $D = 128$:

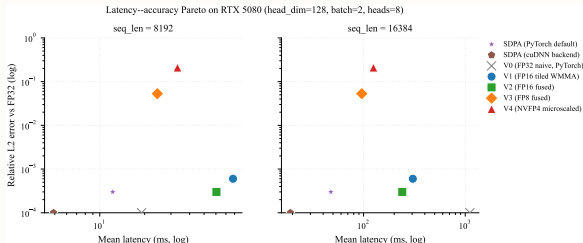
- ▶ V2: ~ 60 ms
- ▶ V3: ~ 26 ms ($2.4\times$ V2)
- ▶ SDPA: ~ 12 ms

Aha moment

Why V3 wins this much:

- ▶ FP8 mma throughput ($\sim 2\times$ FP16)
- ▶ Register-resident O (V2 round-trips through SMEM)

Both unlocked by the same change:
PTX-specified fragment layout.



- ▶ V4 (red triangle) sits at **higher latency** and **higher error** than V3 (orange diamond). Strictly dominated.
- ▶ On consumer Blackwell with our software-microscaled path, FP4's hardware throughput advantage is more than cancelled by the per-mma scaling overhead.

Spoiler

This does *not* contradict SageAttention3's positive FP4 result. Their hardware path (mxsf4nvf4.m16n8k64) internalises the per-element scale at the mma boundary. **The blocker is the layout, not the FP4 idea.**

What we expected

PyTorch SDPA on RTX 5080 dispatches to **FlashAttention-2 in FP16**. Beating it would mean beating a heavily-tuned FA-2 kernel.

What broke

The PyTorch 2.9.1+cu128 wheel was compiled *without* flash attention. cuDNN attention and memory-efficient attention are runtime-disabled for our (sm_120, FP16, non-causal) inputs. SDPA falls through to the **MATH backend** – a pair of cuBLAS matmuls with a softmax in between. It materialises the full S and P .

Concretely, when we tried to pin the dispatch:

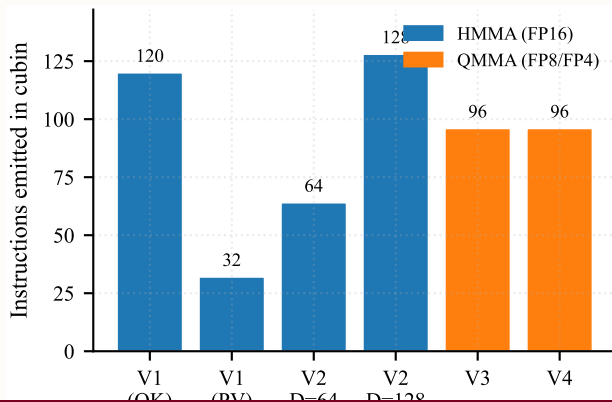
```
with torch.nn.attention.sdpa_kernel(SDPBackend.FLASH_ATTENTION):  
    F.scaled_dot_product_attention(Q, K, V) # raises NoBackendError
```

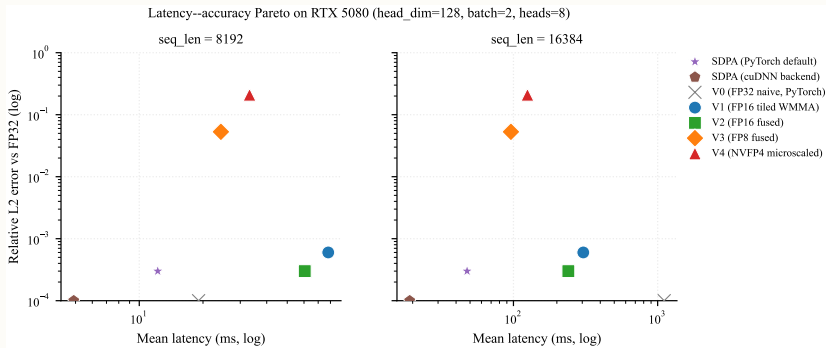
TL;DR

Comparing a custom kernel “against SDPA” without naming the dispatch is misleading. Always pin the backend; always disclose it.

Nsight Compute hardware counters require admin (ERR_NVGPUCTRPERM); on a consumer card without admin we cannot use them. **So we read the SASS directly.**

```
cuobjdump --dump-sass kernels/v3_flash_fp8/_C.cp311-win_amd64.pyd '  
| Select-String QMMA | Measure-Object | % Count  
# 96
```





- **V3 is the practical sweet spot** when an application can absorb $\sim 5\%$ relative L2 error.
- SDPA is faster but has a hidden cost (no fusion on this stack; full S, P in HBM).
- V4 sits dominated by V3 in this software stack – waiting for the hardware-microscaling path.

Part 6 of 6

What to take home

*Take-aways, limitations,
and git clone.*

1. **Algorithmic fusion is the biggest single win.** The $17\times$ memory reduction at $V1 \rightarrow V2$ came from changing the structure, not the precision. Always pull the memory lever first.
2. **Narrow precision pays off only when the layout is knowable.** The $V2 \rightarrow V3$ latency win is roughly half FP8 throughput, half register-resident O. Both required the fragment layout to be specified by the PTX ISA.
3. **Lower precision is not strictly better.** FP4 lost on consumer Blackwell because the cheap-microscaling path is incompatible with the SMEM layout we share with the FP8 kernel. *Hardware quirks defeat “obvious” optimisations.*
4. **Vendor baselines are moving targets.** The SDPA dispatch is build- and OS-dependent. Pin it, disclose it, measure it.
5. **Read the SASS.** `cuobjdump` tells you exactly which Tensor-Core instructions you emitted – more reliable than runtime metrics gated by admin permissions.

- ▶ **Hardware-microscaled NVFP4** (`mx4nvp4.m16n8k64`). Would likely flip V4 vs V3. Requires a swizzled SMEM layout incompatible with V3's idiom; multi-week scope.
- ▶ **Backward pass.** Non-trivial for narrow precisions (Attn-QAT documents specific instabilities). Forward inference only here.
- ▶ **RTX 5090 replication.** Same compute capability; very different SM count, memory bus, VRAM. Would tell us how much of the V4 result is hardware-microscaling- availability vs 5080-specific.
- ▶ **Backend pinning that actually works.** The SDPA FLASH_ATTENTION pin failed on our build; on a Linux build with a flash-enabled wheel it would not. A one-day Linux re-run remains.
- ▶ **Production-grade tile tuning.** Our 64×64 tile is a defensible default; an autotuned sweep over tile sizes and warps-per-block is deferred.

```
# Build the kernels (once per machine):
. .\env\activate_for_build.ps1
pip install --no-build-isolation -e kernels/v0_naive_fp32
pip install --no-build-isolation -e kernels/v1_tiled_fp16
pip install --no-build-isolation -e kernels/v2_flash_fp16
pip install --no-build-isolation -e kernels/v3_flash_fp8
pip install --no-build-isolation -e kernels/v4_flash_nvfp4

# Run the full sweep + figures:
python bench/run_all.py --config bench/configs/sweep_final.yaml '
    --output bench/results/sweep_final.parquet
python analysis/paper_figures.py
```

Aha moment

What you get:

- ▶ One Parquet file with every per-iteration latency + full provenance (driver, CUDA, Python, PyTorch).
- ▶ Six paper-grade figures and a LaTeX results table.
- ▶ 495 passing pytest correctness tests across V0–V4.

Questions?

Code, paper, and slides:

`github.com/canoztas/cmp674-attention-blackwell`

Refik Can Öztaş

N25142279

Ali Talip Coşkun

N25123060

CMP 674 – Parallel Computing with GPUs – Hacettepe University